

Data Lakehouse

Project Setup & Completion Guide

Trino + Iceberg + ClickHouse + MinIO

*This guide walks through everything needed to get the full stack running.
Both frontend UIs are already built. Focus is on infrastructure and data seeding.*

. Table of Contents

- 1 Project Overview
- 2 Prerequisites
- 3 Quick Start - Docker Compose
- 4 Step 1: Fix MinIO Bucket Permissions
- 5 Step 2: Seed ClickHouse with Sample Data
- 6 Step 3: Seed Iceberg Tables via Trino
- 7 Step 4: Run the Frontend UIs
- 8 Port Reference
- 9 Architecture Diagram
- 10 Troubleshooting
- 11 Next Steps & Ideas

1. Project Overview

This is a modern data lakehouse platform that combines a data lake (MinIO / S3) with Apache Iceberg (for ACID transactions on the lake), Trino (for federated SQL queries across sources), and ClickHouse (for fast columnar analytics).

The project has two frontend applications:

- data-product-ui -- A sales analytics dashboard showing revenue, units sold, sales per day, sales by category, and top products. Queries ClickHouse views in the ecommerce_product database.
- query-explorer-ui -- A federated SQL query explorer with a catalog/schema/table browser and a CSV upload tool that ingests data into ClickHouse via S3.

Both UIs are React + Vite apps. Source code, dependencies (node_modules), and production builds (dist/) are all present. No frontend coding is needed.

Services

- MinIO (ports 9000, 9001) -- S3-compatible object storage. Root bucket: warehouse
- PostgreSQL (port 5432) -- Stores Iceberg catalog metadata (database: iceberg_catalog)
- Trino (port 8080) -- Open-source distributed SQL query engine
- ClickHouse (port 8123) -- Columnar analytics database
- mc -- Init container that creates the warehouse bucket on MinIO at startup

2. Prerequisites

Make sure you have the following installed before starting:

- Docker Desktop (or Docker Engine + Docker Compose plugin)
- Node.js v18+ and npm (for running the frontend UIs outside Docker)
- A web browser (Chrome / Firefox / Edge)
- At least 8 GB of RAM allocated to Docker
- Git (optional, for version control)

To verify Docker is ready, run:

```
docker compose version
docker info
```

3. Quick Start - Docker Compose

From the project root directory, start all services with:

```
docker compose up -d
```

This launches MinIO, PostgreSQL, Trino, ClickHouse, and the mc init container.

Check that all services are running:

```
docker compose ps
```

Wait 20-30 seconds for services to initialize (especially Trino, which takes time to load connectors). Then check logs for any errors:

```
docker compose logs --tail=50 trino
```

Note: If a port is already in use, edit docker-compose.yml and change the left side of the port mapping (e.g., '8081:8080' -> '8082:8080').

4. Step 1: Fix MinIO Bucket Permissions

The current docker-compose.yml creates the warehouse bucket with read-only public access. However, the CSV upload tool in query-explorer-ui needs to write files to MinIO. You need to update the mc init container command.

In docker-compose.yml, find the mc service and change its command to:

```
command:
  - /bin/sh
  - -c
  - |
    sleep 5
    mc alias set myminio http://minio:9000 minioadmin minioadmin
    mc mb myminio/warehouse --ignore-existing
    mc anonymous set public myminio/warehouse
    mc admin policy set myminio readwrite user=minioadmin
```

The key thing: the 'warehouse' bucket must allow write access for the CSV upload feature to work. The simplest fix is to use authenticated read/write permissions.

5. Step 2: Seed ClickHouse with Sample Data

The data-product-ui dashboard queries three ClickHouse views in the ecommerce_product database:

- ecommerce_product.daily_sales
- ecommerce_product.sales_by_category
- ecommerce_product.top_products

These tables/views do not exist yet. You must create them. The easiest way is to create an init SQL file that ClickHouse will run automatically on first startup.

What to do:

1. Create a folder called clickhouse-init at the project root.
2. Inside it, create a file called init.sql with the following content:

```
CREATE DATABASE IF NOT EXISTS ecommerce_product;

USE ecommerce_product;

CREATE TABLE IF NOT EXISTS raw_sales (
    sale_date Date,
    product_name String,
    category String,
    quantity UInt32,
    unit_price Float64,
    total_amount Float64
) ENGINE = MergeTree()
ORDER BY (sale_date, category);

INSERT INTO raw_sales VALUES
('2025-01-01', 'Wireless Mouse', 'Electronics', 15, 29.99, 449.85),
('2025-01-01', 'Desk Lamp', 'Home & Office', 8, 45.00, 360.00),
('2025-01-02', 'USB-C Hub', 'Electronics', 22, 34.99, 769.78),
('2025-01-02', 'Notebook Set', 'Stationery', 30, 12.50, 375.00),
('2025-01-03', 'Mechanical Keyboard', 'Electronics', 10, 89.99, 899.90),
('2025-01-03', 'Standing Desk', 'Furniture', 3, 399.00, 1197.00),
('2025-01-04', 'Monitor Arm', 'Office', 12, 79.99, 959.88),
('2025-01-04', 'Coffee Mug', 'Kitchen', 50, 14.99, 749.50),
('2025-01-05', 'Webcam HD', 'Electronics', 18, 59.99, 1079.82),
('2025-01-05', 'Desk Organizer', 'Home & Office', 25, 22.99, 574.75);

CREATE MATERIALIZED VIEW IF NOT EXISTS daily_sales
ENGINE = SummingMergeTree()
ORDER BY sale_date
POPULATE AS
SELECT
    sale_date,
    SUM(quantity) AS units_sold,
    SUM(total_amount) AS revenue
FROM raw_sales
GROUP BY sale_date;

CREATE MATERIALIZED VIEW IF NOT EXISTS sales_by_category
```

```
ENGINE = SummingMergeTree()
ORDER BY category
POPULATE AS
SELECT
  category,
  SUM(quantity) AS units_sold,
  SUM(total_amount) AS revenue
FROM raw_sales
GROUP BY category;

CREATE MATERIALIZED VIEW IF NOT EXISTS top_products
ENGINE = SummingMergeTree()
ORDER BY total_revenue DESC
POPULATE AS
SELECT
  product_name,
  SUM(quantity) AS units_sold,
  SUM(total_amount) AS total_revenue
FROM raw_sales
GROUP BY product_name
ORDER BY total_revenue DESC;
```

3. Add a volume mount for this folder in the clickhouse service in docker-compose.yml:

```
services:
  clickhouse:
    ...
    volumes:
      - clickhouse_data:/var/lib/clickhouse
      - ./clickhouse-init:/docker-entrypoint-initdb.d # ADD THIS LINE
```

4. Restart ClickHouse to run the init script:

```
docker compose down clickhouse
docker compose up -d clickhouse
```

ClickHouse auto-executes .sql files in /docker-entrypoint-initdb.d/ on first startup. If the clickhouse_data volume already exists, delete it first

6. Step 3: Seed Iceberg Tables via Trino

The Iceberg catalog is empty -- no tables exist in the `s3://warehouse/` location. You can create sample Iceberg tables by connecting to Trino and running DDL statements.

Connect to Trino

Use the Trino CLI (inside Docker) or any JDBC client:

```
# Option 1: Trino CLI in Docker
docker exec -it data-lakehouse-trino-1 trino

# Option 2: Use the web UI at http://localhost:8080/ui
```

Create Sample Iceberg Tables

Once connected to Trino, run these SQL statements:

```
CREATE SCHEMA IF NOT EXISTS iceberg.lakehouse
WITH (location = 's3://warehouse/lakehouse');

CREATE TABLE IF NOT EXISTS iceberg.lakehouse.products (
  product_id BIGINT,
  name VARCHAR,
  category VARCHAR,
  price DOUBLE,
  created_at TIMESTAMP
) WITH (
  format = 'PARQUET',
  location = 's3://warehouse/lakehouse/products'
);

INSERT INTO iceberg.lakehouse.products VALUES
(1, 'Wireless Mouse', 'Electronics', 29.99, TIMESTAMP '2025-01-01 00:00:00'),
(2, 'Desk Lamp', 'Home & Office', 45.00, TIMESTAMP '2025-01-01 00:00:00'),
(3, 'USB-C Hub', 'Electronics', 34.99, TIMESTAMP '2025-01-02 00:00:00'),
(4, 'Mechanical Keyboard', 'Electronics', 89.99, TIMESTAMP '2025-01-03 00:00:00'),
(5, 'Standing Desk', 'Furniture', 399.00, TIMESTAMP '2025-01-03 00:00:00');

SELECT * FROM iceberg.lakehouse.products;
```

After seeding, you can explore these tables via the query-explorer-ui at <http://localhost:5173>.

7. Step 4: Run the Frontend UIs

Each UI is a separate Vite dev server. Run them in two terminals.

Data Product UI (Dashboard)

```
cd data-product-ui  
npm run dev
```

Opens at <http://localhost:5174>

This dashboard shows sales metrics from ClickHouse. ClickHouse must be running and seeded with data for this to work.

Query Explorer UI

```
cd query-explorer-ui  
npm run dev
```

Opens at <http://localhost:5173>

This has two tabs: a SQL query editor (connects to Trino at port 8080) and a CSV upload tool (uploads to MinIO and ingests into ClickHouse).

Production Builds

Both apps already have production builds in their `dist/` folders. You can serve them with any static file server (e.g., `nginx`, `serve`).

```
# Option: serve production build  
npx serve data-product-ui/dist -l 4173  
npx serve query-explorer-ui/dist -l 4174
```

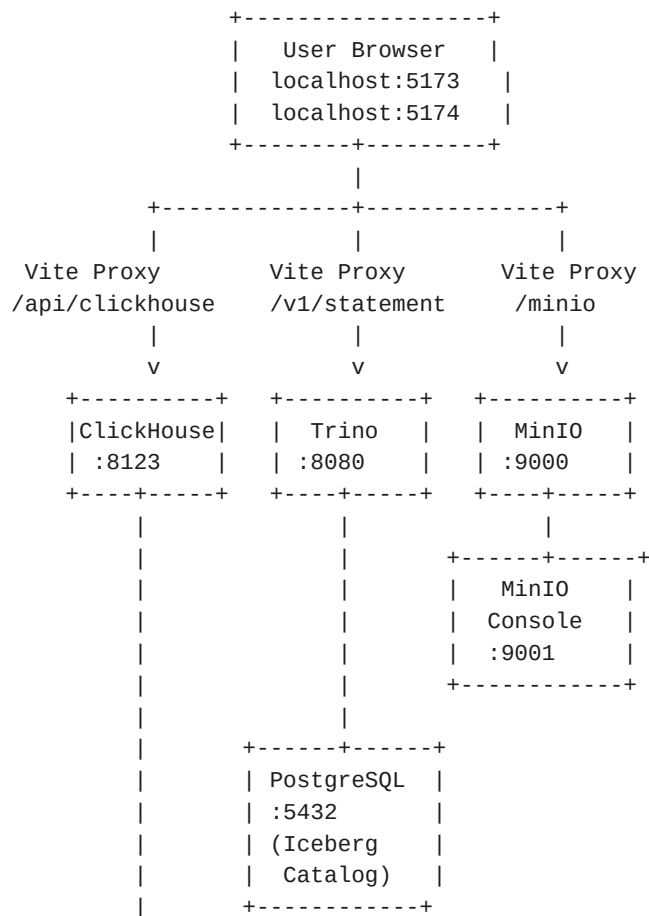
8. Port Reference

Here is a quick reference for all the ports used by the project:

5173	Query Explorer UI (dev server)
5174	Data Product UI (dev server)
8080	Trino (REST API & web UI at /ui)
5432	PostgreSQL (Iceberg catalog)
9000	MinIO S3 API
9001	MinIO Console (web UI)
8123	ClickHouse HTTP interface

9. Architecture Diagram

Below is a textual representation of the system architecture:



10. Troubleshooting

ClickHouse views return no data

The ecommerce_product database and its views need to be created. Follow Step 2 to create the init SQL script and mount it into the ClickHouse container.

MinIO upload fails (403)

The warehouse bucket has read-only public access. Update the mc init command in docker-compose.yml to grant write access. See Step 1.

Trino shows no Iceberg tables

The PostgreSQL catalog database might not be initialized. Check that postgres is running and Trino can connect: `docker compose logs trino | grep iceberg`.

Port already in use

If a port is already taken, change the left side of the port mapping in docker-compose.yml (e.g., '8081:8080' -> '8082:8080').

npm run dev fails

Make sure Node.js v18+ is installed. Delete node_modules and reinstall:

```
cd data-product-ui && rm -rf node_modules && npm install
cd query-explorer-ui && rm -rf node_modules && npm install
```

Docker container keeps restarting

Check logs with `'docker compose logs <service-name>'`. Common causes: missing dependencies, wrong credentials, or port conflicts.

Can't connect to Trino from query-explorer-ui

Make sure the Vite proxy in query-explorer-ui/vite.config.js points to the correct Trino URL. Default is `http://localhost:8080`.

11. Next Steps & Ideas

Once the basic setup is working, here are some ideas for extending the project:

1. Add more sample data - Create larger datasets for both ClickHouse and Iceberg to make the dashboards more interesting.
 2. Set up dbt - Use dbt with the Trino adapter to manage data transformations and materialized views in a more production-like workflow.
 3. Add Superset or Grafana - Connect Apache Superset or Grafana to Trino or ClickHouse for richer visualizations beyond the built-in UIs.
 4. Automate with orchestration - Use Apache Airflow or Dagster to schedule data ingestion and transformation pipelines.
 5. Add a third UI - Build a simple Streamlit or Dash app that queries both Iceberg (via Trino) and ClickHouse for comparison.
 6. Containerize the UIs - Create Dockerfiles for the two React apps and add them to docker-compose.yml so everything runs with one command.
 7. Add CI/CD - Set up GitHub Actions to lint, test, and build the UIs on every push.
 8. Explore Iceberg features - Try time travel queries, schema evolution, and partition evolution using Trino's Iceberg connector.
-

Good luck! If you get stuck, check the docker compose logs for each service.

The project root is: E:\New folder (25)\data-lakehouse